

Language Agent Tree Search (LATS)

NeurIPS 2026 Education Track

Michael Li, Eric Bae, Tianyi Huang

Lecture Goals

Part 1

Prerequisites

- This lecture is designed for *advanced undergraduates or first-year graduates*
- Entering this lecture, you should already have...
 - Intermediate proficiency in Python coding
 - Basic knowledge of statistics and computer algorithms
 - Basic knowledge of language model prompting

Learning Objectives

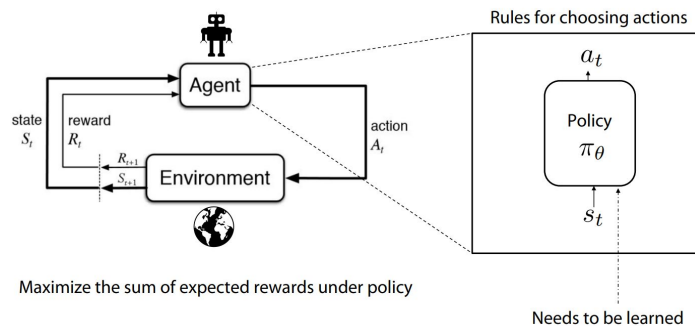
- By the end of this lecture, you should be able to...
 - *Understand* why LATS is preferred over linear reasoning methods and sequential agent loops in complex environments
 - *Recall* the 6 core operations in LATS (selection, expansion, evaluation, simulation, backpropagation, reflection) and how they work
 - *Trace* LATS execution to explain why specific reasoning paths were taken and why decisions were made
 - *Assess* the practical trade-offs of LATS, specifically regarding token consumption, latency, and API costs versus gains in reasoning

LATS Technicals

Part 2

What is an Agent?

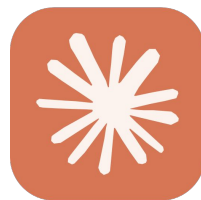
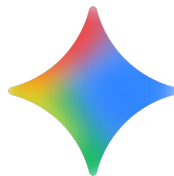
- An **agent** is policy which takes actions in an environment
 - **Environment** = the world the agent acts in
 - **Policy** = the rules the agent follows
 - **State** = the complete description of the world at a timestep
 - **Observation** = what the agent sees at a timestep
 - **Action** = the move an agent takes at a timestep
 - **Reward** = the feedback returned to the agent



Note that this is the same notation as in reinforcement learning!

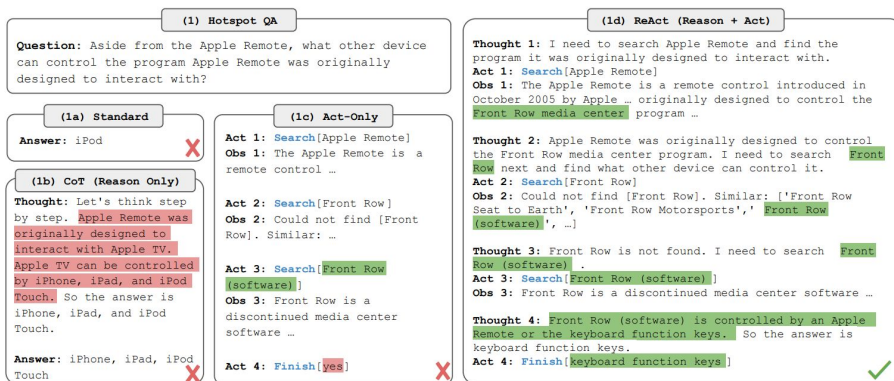
What is an LLM Agent?

- An **LLM agent**, also known as a “language agent”, is an agent that uses an LLM as part of its policy (i.e. to help determine actions)
- What can an LLM agent do?
 - Formulate plans and executes multi-step operations
 - Observe, orient, decide, and act in continuous loops
 - Retrieve up-to-date world information dynamically
 - Interact with the world via function calling and tools



ReAct

- **ReAct** is a foundational agent paradigm interleaving reasoning with actions
 - The agent receives a real-world observation, generates a thought about the current state, uses a tool to take an action, and repeats
- Limitations
 - Errors compound over multiple steps of reasoning and action
 - ReAct follows a single unbranching path, it cannot explore different paths without entirely starting over again



Upper Confidence Bound

- **Upper confidence bound (UCB)** is an algorithm used to balance exploration and exploitation in uncertain environments
 - **Exploration** = trying out paths which we do not have much data on
 - **Exploitation** = choosing paths that we already know work well
- Compute UCB for all actions, and take the action with the highest UCB
 - Intuition: unexplored actions start with high UCB, but if we try it and discover that it is bad, its UCB drops

Empirical reward of the action so far

$$UCB = V_i + C \sqrt{\frac{\ln N}{n_i}}$$

Exploration factor

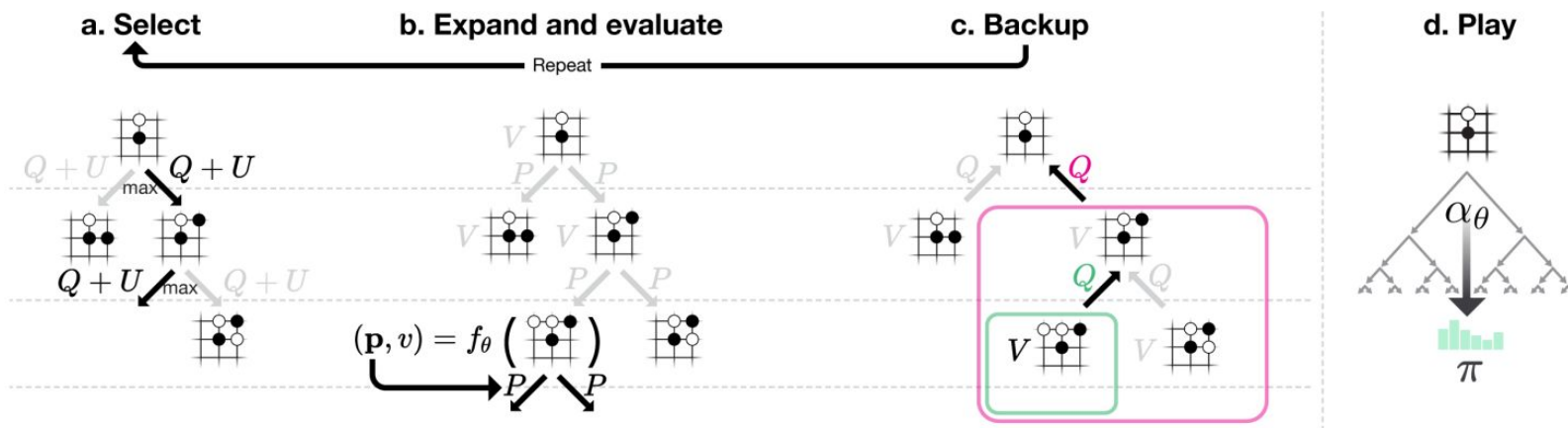
Monte Carlo Tree Search

- **Monte Carlo tree search** (MCTS) is a planning algorithm that searches for optimal decisions using random sampling and tree building
- MCTS has 4 steps:
 - (1) **Selection:** Use UCB to navigate down the tree to a leaf node
 - (2) **Expansion:** Add new child nodes (potential next actions) to that node
 - (3) **Simulation:** Run a “rollout”, or simulating random steps to the end
 - (4) **Backpropagation:** Pass the simulation score back up the tree to update the value of all previous nodes
- Over time, MCTS concentrates sampling on the most promising paths

Note that technically speaking, MCTS uses UCT (UCB for trees)

Monte Carlo Tree Search

- MCTS is used extensively in situations where the search space is too big for exact search algorithms that look at every case
- AlphaGo is probably the most famous application of MCTS
 - MCTS is also widely used in areas such as trajectory planning

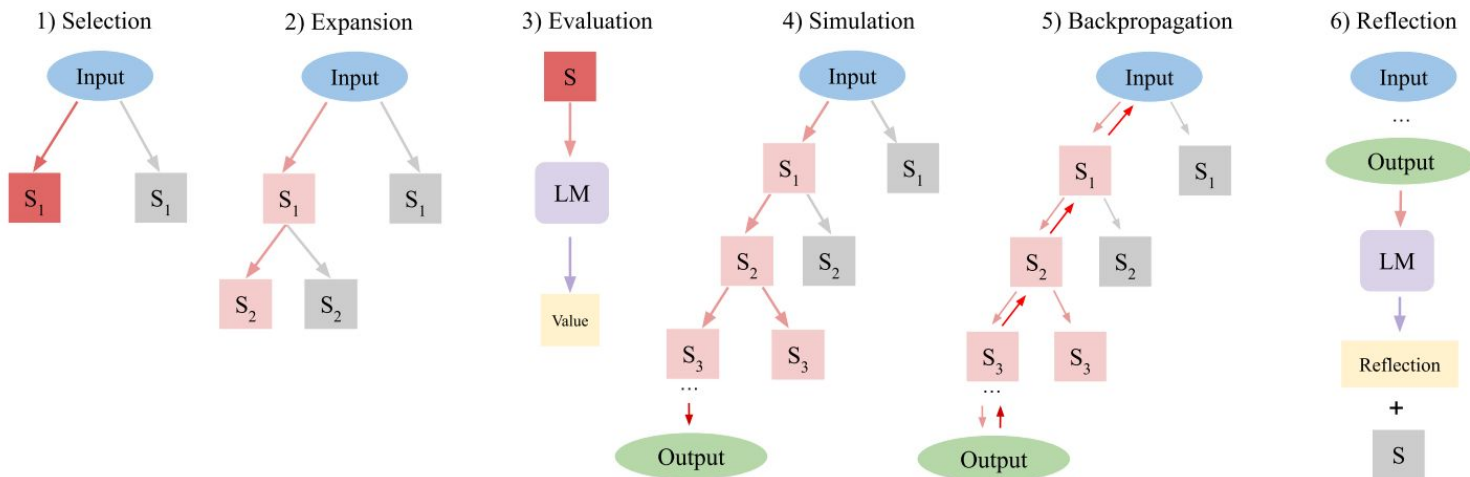


Language Agent Tree Search

- **Language agent tree search** (LATS) applies MCTS to help LLM agents plan
- LATS adds two new operations on top of MCTS
 - **Evaluation:** The LLM acts as an evaluator and assigns a scalar score to each new state to quantify its progress toward the goal
 - **Reflection:** When an action leads to a failed or suboptimal trajectory, the LLM generates a verbal self-critique
- Evaluation is needed because LLM agents cannot easily “rollout” to the end
- Reflection is needed because it provides a useful semantic signal

Language Agent Tree Search

- The full LATS workflow (not in order, grouped by step similarity):
 - **Selection:** Use UCB to navigate down the tree to a leaf node
 - **Expansion & Simulation:** Generate and execute next actions
 - **Evaluation & Reflection:** Score and analyze feedback with LLM
 - **Backpropagation:** Pass the simulation score back up the tree



LATS Pseudocode

- This is what LATS looks like from a high level, in implementation
- Our repository has a complete and fully functional LATS example
- https://github.com/Ryquo-Labs/LATS-Visualizer/blob/main/scripts/run_lats/search.py

```
def LATS(x, k=5, n_iter=50, w=1.0):
    root, reflections = Node(state=x), []
    for _ in range(n_iter):
        node = select(root, w)
        children = expand(node, k, reflections)
        for c in children:
            c.score = LM_value(c, reflections)
        best = argmax(children, key=lambda c: c.score)
        r, solved, traj = simulate(best, reflections)
        backpropagate(best, r)
        if solved:
            return traj
        reflections.append(LM_reflect(traj, r))
    return best_trajectory(root)

def select(node, w):
    """Greedy descend the tree by UCT until reaching an unexpanded leaf."""
    while node.children:
        node = argmax(node.children, key=lambda c: UCT(c, w))
    return node

def UCT(c, w):
    return c.V / c.N + w * sqrt(log(c.parent.N) / c.N) # exploit + explore

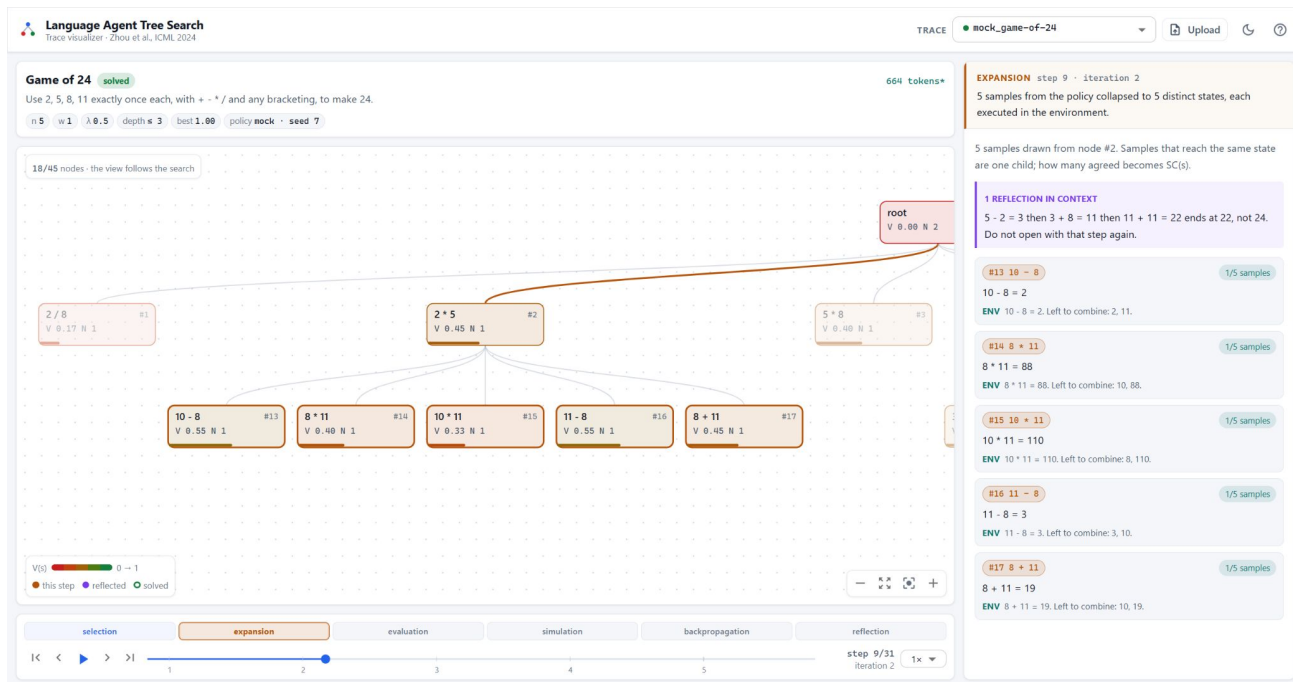
def expand(node, k, reflections):
    """Sample k candidate actions a ~ p_LM(. | s, reflections) and execute each."""
    node.children = []
    for a in LM_sample_actions(node.state, reflections, n=k):
        o, r, done = env.step(node.state, a)
        node.children.append(Node(state=node.state + [a, o], parent=node,
                                action=a, reward=r, done=done))
    return node.children

def simulate(node, reflections):
    """Roll out greedily with the LM policy until terminal; return final reward."""
    while not node.done:
        node = argmax(expand(node, 1, reflections), key=lambda c: LM_value(c, reflections))
    return node.reward, node.reward >= THRESHOLD, node.state

def backpropagate(node, r):
    while node is not None:
        node.N += 1
        node.V += r
        node = node.parent # value = mean return
```

LATS Visualization

- <https://ryquo-labs.github.io/LATS-Visualizer/>
- Let's step through a few LATS executions!



LATS in the Real World

Part 3

Performance

- LATS does well on HotpotQA with both reasoning-based prompting strategies and acting-based prompting strategies

Prompt Method	HotpotQA (EM) $\uparrow$
Base LM	0.32
CoT (Wei et al., 2022)	0.34
CoT - SC (Wang et al., 2022)	0.38
ToT (Yao et al., 2023a)	0.55
RAP (Hao et al., 2023)	0.60
RAP ($n = 10$)	0.60
LATS (CoT)	0.62

Table 2. GPT-3.5 reasoning-based prompting results on HotpotQA. LATS achieves the highest exact match (EM) for reasoning. We sample $n = 5$ nodes during expansion and $k = 50$ trajectories.

Prompt Method	HotpotQA (EM) $\uparrow$
ReAct (Yao et al., 2023b)	0.32
ReAct (best of k)	0.38
Reflexion (Shinn et al., 2023)	0.51
ToT (ReAct)	0.39
RAP (ReAct)	0.54
LATS (ReAct)	0.63
LATS ($n = 3$)	0.58
LATS ($n = 10$)	0.65
LATS (CoT + ReAct)	0.71

Table 3. GPT-3.5 acting-based prompting results on HotpotQA. LATS achieves the highest exact match (EM) for acting. We sample $n = 5$ nodes and use $k = 50$ trajectories. We also evaluate sampling ReAct k times and using both CoT and ReAct base prompting designs for LATS, which achieves the best performance. Note that LATS outperforms ToT and RAP with ReAct prompting, which are the simple adaptations of search algorithms to decision-making.

Performance

- LATS outperforms linear methods like ReAct and Reflexion on benchmarks

Method	Score $\uparrow$	SR $\uparrow$
ReAct (Yao et al., 2023b)	53.8	28.0
ReAct (best of k)	59.1	32.0
Reflexion (Shinn et al., 2023)	64.2	35.0
LATS (ReAct)	75.9	38.0
IL (Yao et al., 2022)	59.9	29.1
IL+RL (Yao et al., 2022)	62.4	28.7
Fine-tuning (Furuta et al., 2024)	67.5	45.0
Expert	82.1	59.6

Table 6. Score and success rate (SR) on WebShop. Results are organized into prompting, RL-based training, and human performance. For the same number of iterations, LATS improves both score and SR and surpasses RL-based training.

Prompt Method	Game of 24 (Success Rate) $\uparrow$
CoT (Wei et al., 2022)	0.08
Reflexion (Shinn et al., 2023)	0.12
ToT (Yao et al., 2023a)	0.20
RAP (Hao et al., 2023)	0.40
LATS (CoT)	0.44

Table 7. Results on Game of 24 with GPT-3.5. We sample $n = 5$ nodes and $k = 30$ trajectories.

Performance

- LATS ablations show that all components are necessary for best performance

Prompt Method	HotPotQA (EM) $\uparrow$
ToT (ReAct)	0.39
RAP (ReAct)	0.54
LATS (No LM Heuristic)	0.37
LATS (DFS)	0.42
LATS (No Reflection)	0.58
LATS (ReAct)	0.63

Table 8. Ablation results on LATS and baseline variants in HotPotQA. We use ReAct as the base prompt and sample $n = 5$ children and $k = 50$ trajectories. LATS requires every component and operation for optimal performance.

Performance

- However, the increased performance offered by LATS has a direct tradeoff with speed and compute
 - Tree expansion requires significantly more API calls

Method	Performance $\uparrow$	Sample complexity $\downarrow$	Token Consumption $\downarrow$
ReAct (Best $k = 250$)	0.42	$O(k)$	-
CoT-SC ($n = 1, k = 250$)	0.40	$O(k)$	-
LATS ($n = 1, k = 50$)	0.48	$O(k)$	-
ToT (ReAct, $n = 5, k = 50$)	0.49	$O(kn)$	210, 215
RAP (ReAct, $n = 5, k = 50$)	0.54	$O(kn)$	176, 500
LATS ($n = 5, k = 50$)	0.63	$O(kn)$	173, 290

Table 9. Performance, sample complexity of different methods, average number of nodes expanded, and token consumption upon success by methods with tree-based search. n is the number of children nodes expanded at every step and k is the number of trajectories. LATS has the same sample complexity as other methods with tree-based search and expands less nodes upon success, which indicates lower token cost.

Application Scenarios

- *LATS trades execution efficiency for reasoning depth, making it ideal for complex or high-stakes tasks!*
- LATS also requires objective, external feedback to guide its search
- Great places to use LATS include...
 - Code generation
 - Computer use
 - Web navigation
 - Multi-hop QA
 - ...

Learning Objectives Answered

- By the end of this lecture, you should be able to...
 - *Understand* why LATS is preferred over linear reasoning methods and sequential agent loops in complex environments
[uses recoverable errors rather than compounding ones]
 - *Recall* the 6 core operations in LATS (selection, expansion, evaluation, simulation, backpropagation, reflection) and how they work
[4 MCTS steps + LLM evaluation and reflection]
 - *Trace* LATS execution to explain why specific reasoning paths were taken and why decisions were made
[UCB, LLM value score, reflection → which branch, node, region]
 - *Assess* the practical trade-offs of LATS, specifically regarding token consumption, latency, and API costs versus gains in reasoning
[LATS is less efficient but performs better than linear strategies]